Sesión 08-A: Pipeline CNN Completo — Del Problema a la Arquitectura

Integración de la Unidad I (De la exploración al modelo profesional)

IS056B Ingeniería del Conocimiento
Msc. Jaime Antonio Huaytalla Pariona
Universidad Nacional del Centro del Perú (UNCP) 2026-II

El Pipeline Profesional: De la Exploración a la Ingeniería



Notebook Exploratorio	Pipeline Profesional
Celdas sueltas (prueba y error)	**Modular**, **funciones/clases** organizadas
Depende del orden de ejecución	**Semilla fija**, **scripts ejecutables** de inicio a fin
Datos cargados manualmente	**Pipeline automatizado** y **validado**
Un solo intento de entrenamiento	**Experimentos versionados** con **logging**
Accuracy básico	**Suite de métricas** + **análisis de errores**

Fase 1: Definición del Problema

1. **¿Qué problema resuelvo?** (Ej. Detectar cultivos, radiografías).
2. **¿Es clasificación binaria o multiclase?** (Define función de pérdida y capa de salida).
3. **¿Cuál es la métrica de éxito?** (Accuracy, F1-Score, Precisión, AUC).
4. **¿Cuáles son las restricciones?** (Tiempo de inferencia, tamaño en MB, hardware).
5. **¿Cuál es la línea base (baseline)?** (Rendimiento de un clasificador trivial).

Case Study Card

Ejemplo Práctico: Enfermedades en Hojas de Plantas

Tipo:	**Clasificación multiclase** (38 categorías, 14 especies).
Dataset:	**PlantVillage** (54,305 imágenes)
Métrica:	**Macro F1-Score** (mitiga el impacto del desbalance).
Baseline:	**2.6% accuracy** (clasificador 100% aleatorio: 1/38).
Restricción:	**Modelo < 50MB** (despliegue en dispositivos móviles agrícolas).

Fase 2: Selección y Validación del Dataset

Repositorios Recomendados

- **kaggle.com/datasets**
- **tensorflow.org/datasets**
- **huggingface.co/datasets**
- **universe.roboflow.com**

Criterios Mínimos de Aceptación

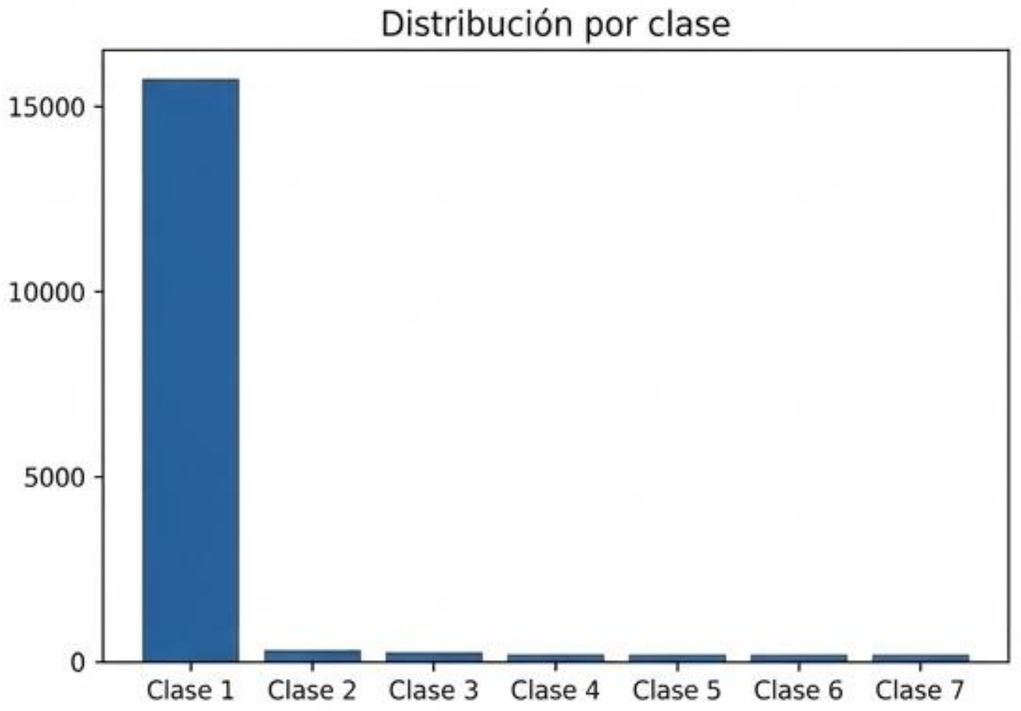
- ✓ **Volumen:** ≥ 1,000 imágenes en total.
- ✓ **Complejidad:** ≥ 3 clases diferentes.
- ✓ **Viabilidad:** Contexto claro y aplicabilidad al mundo real.
- ⊘ **Exclusiones:** Prohibido usar MNIST o Fashion-MNIST (insuficientes para un pipeline completo).

EDA para Imágenes: Distribución de Clases

```python
# Contar imágenes por clase y visualizar
class_counts = {cls: len(os.listdir(os.path.join(train_dir, cls)))
                for cls in os.listdir(train_dir)}

# Gráfica de barras
plt.bar(class_counts.keys(), class_counts.values())
plt.title('Distribución por clase')
plt.show()

# Cálculo rápido del ratio
max_c, min_c = max(class_counts.values()), min(class_counts.values())
print(f'Ratio de desbalance: {max_c/min_c:.1f}:1')
```



El Riesgo del Desbalance: Un modelo en un dataset severamente desbalanceado puede aprender a predecir solo la clase mayoritaria. Obtendrá un 'alto accuracy' engañoso, pero será inútil en la práctica.

EDA para Imágenes: Propiedades y Calidad

Análisis de Dimensiones y Canales

```python
# Extraer metadatos de muestra con PIL
widths, heights, channels = [],[], []
for img_name in os.listdir(class_path)[:50]:
    img = Image.open(os.path.join(class_path, img_name))
    widths.append(img.width)
    heights.append(img.height)
    channels.append(len(img.getbands()))

print(f'Canales: {Counter(channels)}')
print(f'Aspecto promedio: {np.mean(widths)/np.mean(heights):.2f}')
```

Auditoría de Calidad



Imágenes corruptas: Script de detección usando Image.open().verify().



Duplicados: Uso de funciones hash (hashlib) para colisiones exactas.



Ruido visual: Iluminación inconsistente, desenfoque o marcas de agua que confunden al modelo.

Fase 3: Preprocesamiento con ImageDataGenerator

Regla de Oro: ¡Nunca apliques Data Augmentation al set de validación o prueba!

```
# 1. Train CON Data Augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20, zoom_range=0.15,
    horizontal_flip=True, validation_split=0.2
)

# 2. Validation SIN Augmentation (solo rescale)
val_datagen = ImageDataGenerator(
    rescale=1./255, validation_split=0.2
)

# 3. Generación de flujos (Notar el shuffle)
train_generator = train_datagen.flow_from_directory(
    data_dir, target_size=(224, 224), batch_size=32,
    subset='training', seed=42, shuffle=True
)
val_generator = val_datagen.flow_from_directory(
    data_dir, target_size=(224, 224), batch_size=32,
    subset='validation', seed=42, shuffle=False # < ¡CRÍTICO!
)
```

El Pipeline Moderno con tf.data (Recomendado)

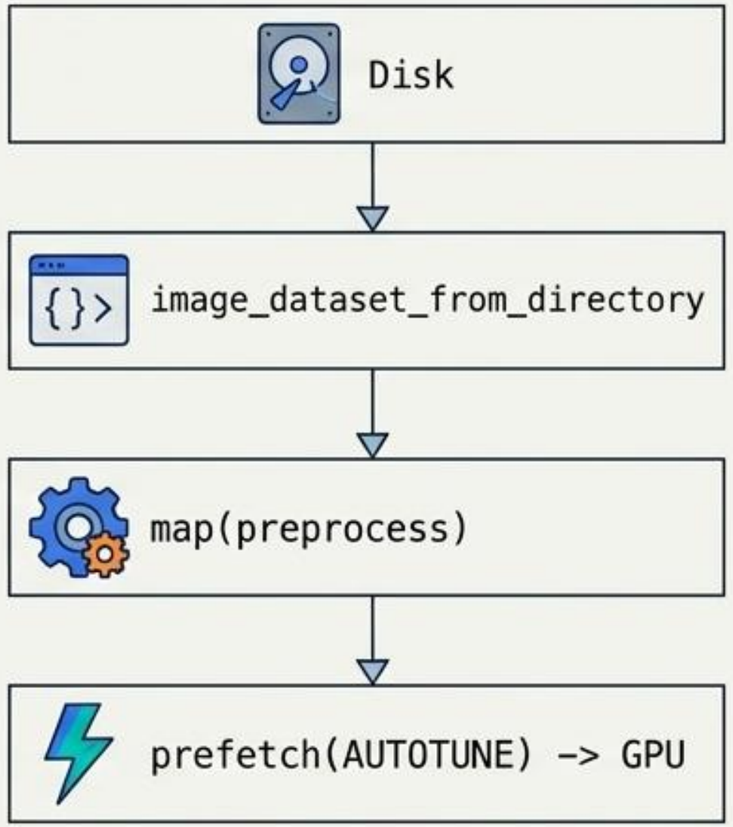
```python
# 1. Carga nativa ultrarrápida
dataset = tf.keras.utils.image_dataset_from_directory(
    directory, image_size=(224, 224), batch_size=32, seed=42
)

# 2. Augmentation como bloque Secuencial
augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip('horizontal'),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1),
])

# 3. Mapeo optimizado con AUTOTUNE
def preprocess(images, labels):
    images = tf.keras.layers.Rescaling(1./255)(images)
    if is_training: images = augmentation(images, training=True)
    return images, labels

dataset = dataset.map(preprocess, num_parallel_calls=tf.data.AUTOTUNE)
dataset = dataset.prefetch(tf.data.AUTOTUNE)
```

Mayor rendimiento, **paralelismo dinámico (AUTOTUNE)** y aumentos como capas nativas.



Guía Estratégica de Data Augmentation

El Principio Rectificador: Toda transformación debe generar una imagen que podría existir en el mundo real.

Técnica	Útil cuando...	Evitar cuando...
Flip Horizontal	**Objetos simétricos (caras, animales)**	**Texto, números, señales de tránsito**
Flip Vertical	**Vistas aéreas, microscopía celular**	Objetos con gravedad fija (personas, autos)
Rotación	Objetos que rotan naturalmente (hojas)	Paisajes de horizonte, documentos fijos. 
Zoom	Existen **diferentes distancias focales** en uso	El dataset ya tiene **multiescala natural**
Brillo / Contraste	**Exteriores, iluminación climática**	Imágenes médicas estrictamente **calibradas**

Fase 4: Modelo 1 — CNN Baseline

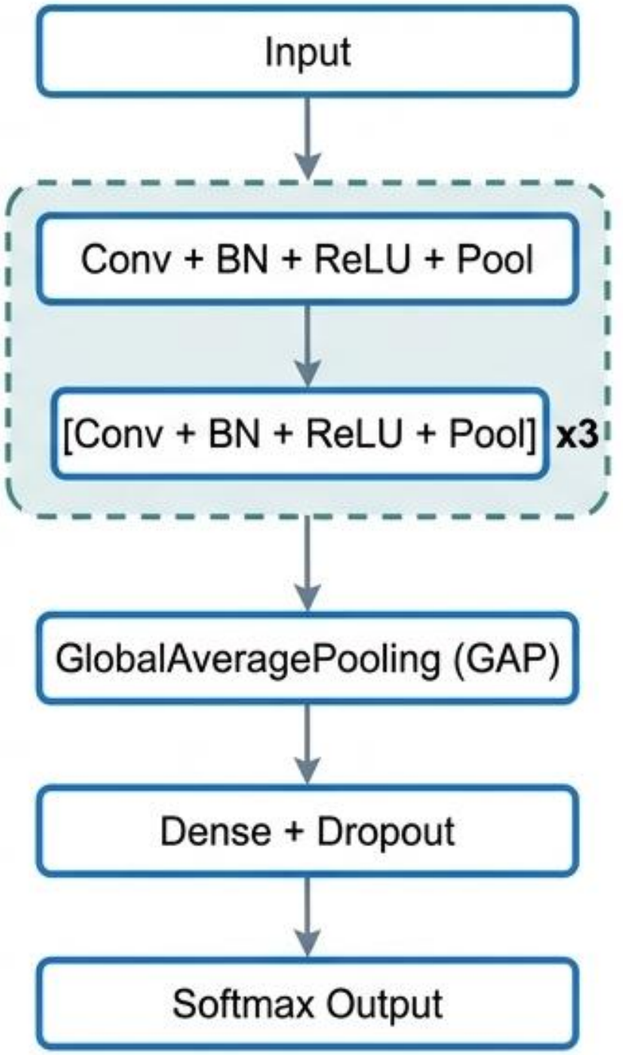
Propósito: Confirmar que el pipeline funciona de principio a fin antes de invertir en arquitecturas complejas.

```python
def build_baseline_model(input_shape, num_classes):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=input_shape),

        # Bloque Conv repetido (32 -> 64 -> 128)
        tf.keras.layers.Conv2D(32, (3, 3), padding='same'),
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.Activation('relu'),
        tf.keras.layers.MaxPooling2D((2, 2)),

        # ... (bloques intermedios omitidos por brevedad) ...

        # Transición y Clasificador
        tf.keras.layers.GlobalAveragePooling2D(),
        tf.keras.layers.Dense(256, activation='relu'),
        tf.keras.layers.Dropout(0.5),
        tf.keras.layers.Dense(num_classes, activation='softmax')
    ])
    return model
```



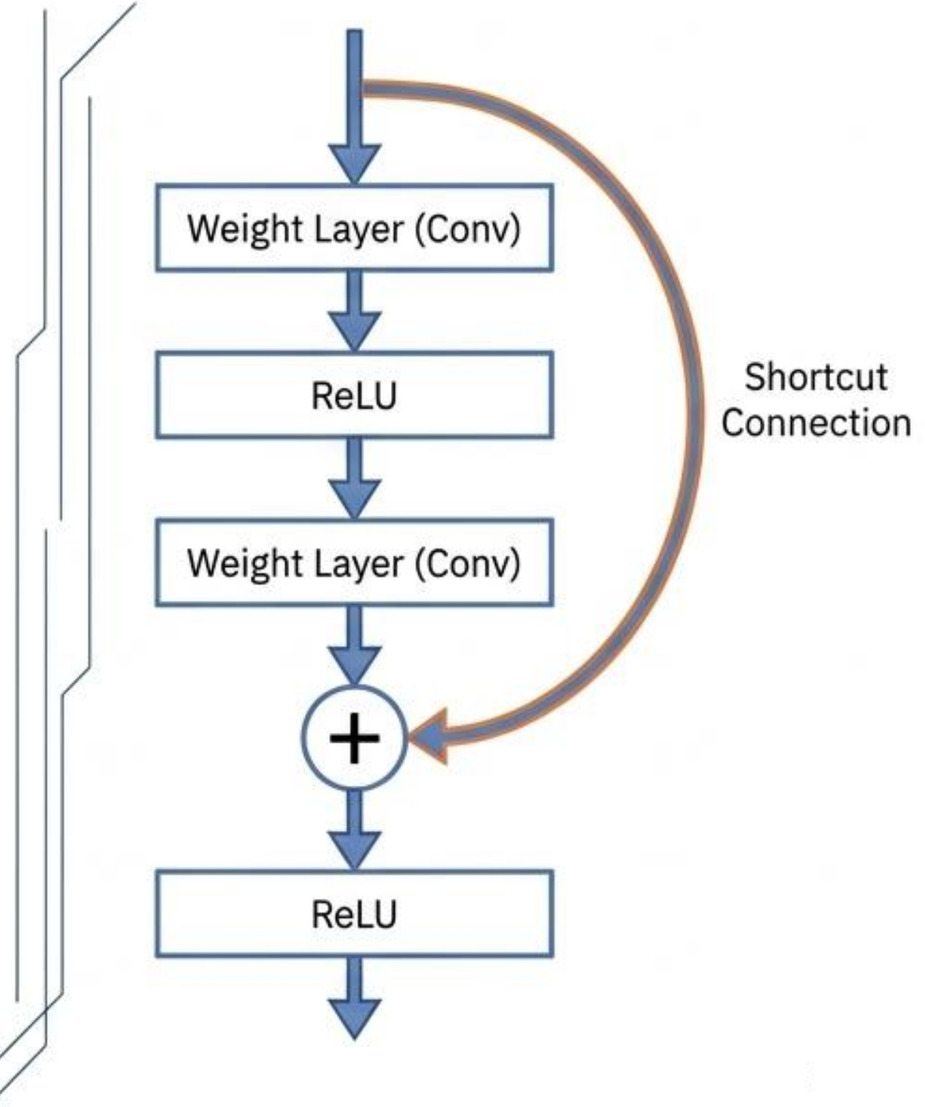
Fase 4: Modelo 2 — CNN Mejorada (ResNet Style)

Implementando Bloques Residuales para combatir el desvanecimiento del gradiente.

```python
def residual_block(x, filters, stride=1):
    shortcut = x

    # Ruta principal
    x = tf.keras.layers.Conv2D(filters, (3,3), strides=stride)(x)
    x = tf.keras.layers.BatchNormalization()(x)
    x = tf.keras.layers.Activation('relu')(x)
    x = tf.keras.layers.Conv2D(filters, (3,3))(x)
    x = tf.keras.layers.BatchNormalization()(x)

    # SUMA (Skip Connection o Atajo)
    x = tf.keras.layers.Add()([x, shortcut])
    return tf.keras.layers.Activation('relu')(x)
```



Fase 4: Modelo 3 — CNN Optimizada Final

Optimizaciones Aplicadas (**S06**)

- **Data Augmentation** nativo incrustado en el grafo.
- **Stem** inicial con **stride agresivo** para **reducción rápida de memoria**.
- **Regularización L2 (1e-4)** en todas las capas densas y convolucionales.
- Múltiples capas de **Dropout** escalonadas para forzar redundancia.

Arquitectura	Parámetros Totales	Parámetros Entrenables	Peso Aprox. (MB)
Modelo 1 (Baseline)	148,000	147,000	~0.6 MB
Modelo 2 (ResBlocks)	2,450,000	2,440,000	~9.5 MB
Modelo 3 (Optimizada)	2,800,000	2,780,000	~10.8 MB

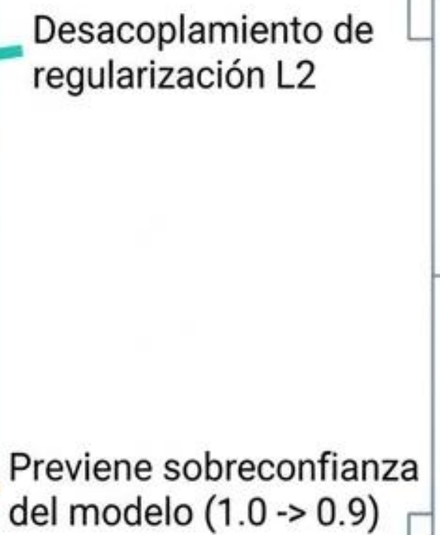
Fase 5: Configuración Centralizada del Entrenamiento

Un solo diccionario como 'Panel de Control' para hiperparámetros.

```python
# Panel de Control Central (CONFIG)
CONFIG = {
    'learning_rate': 1e-3,
    'epochs': 50,
    'batch_size': 32,
    'optimizer': 'adam',
    'weight_decay': 1e-4,
    'label_smoothing': 0.1,
}

# Optimizador y Loss avanzados
optimizer = tf.keras.optimizers.Adam(
    learning_rate=CONFIG['learning_rate'],
    weight_decay=CONFIG['weight_decay']
)
loss_fn = tf.keras.losses.CategoricalCrossentropy(
    label_smoothing=CONFIG['label_smoothing']
)

# Compilación con métricas robustas
model.compile(optimizer=optimizer, loss=loss_fn,
              metrics=['accuracy', tf.keras.metrics.AUC(multi_label=True)])
```



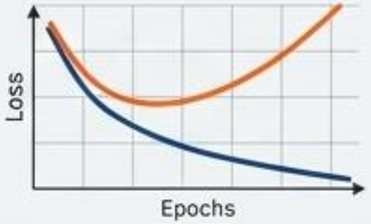
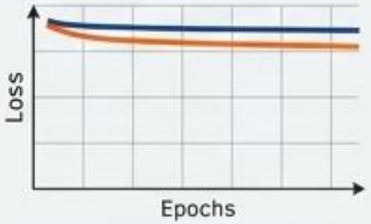
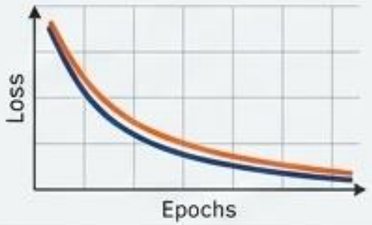
Automatización: Los 5 Callbacks Profesionales

Vigilancia, ajuste y registro automatizado sin intervención humana.

	`ModelCheckpoint`	Guarda empíricamente solo el mejor modelo (save_best_only=True).
	`EarlyStopping`	Evita quemar ciclos y sobreajustar al detener si `val_loss` no mejora.
	`ReduceLROnPlateau`	Micro-ajustes automáticos; reduce el Learning Rate al encontrar mesetas.
	`TensorBoard`	Telemetría interactiva en tiempo real.
	`CSVLogger`	Registro persistente en texto plano para trazabilidad.

Diagnóstico Visual del Entrenamiento

Aprender a "leer" las curvas de aprendizaje (Loss y Accuracy).

Diagnostic Triage Board			
Condition	**Conceptual Graph Icon**	**Symptoms**	**Architectural Treatment**
Overfitting (Sobreajuste)		Brecha expansiva en **Accuracy**. **Memorización** excesiva.	+ **Dropout**, + **Augmentation**, Reducir complejidad.
Underfitting (Subajuste)		Modelo incapaz de capturar patrones básicos.	Modelo más profundo, + **Épocas**, Aumentar **Learning Rate**.
Buen Ajuste (Saludable)		Brecha minúscula y estable.	Dejar actuar al **Early Stopping**.

Próximo Paso (Sesión 08-B): Evaluación rigurosa (ROC, Matriz de Confusión), análisis de errores, versionado técnico y empaquetado para despliegue.